GUIA DO ALUNO

LEDs: do primeiro pisca-pisca ao semáforo controlado pelo Wi-Fi

Lógica de programação com Arduino e ESP32 — LEDs, botões, potenciômetro, LED RGB e página web

---

Placas: Arduino Uno → ESP32 • **Componentes:** LEDs, resistores, botão, potenciômetro, LED RGB

Projeto final: semáforo + LED RGB controlados por uma página web no celular

Carga horária: 4 aulas de 100 min

Espaço Cria — 2026

Sumário

Antes de começar: o que são Arduino e ESP32?

Segurança e cuidados

Aula 1 — Ligar um LED e fazê-lo obedecer

Aula 2 — Mini projeto: semáforo

Aula 3 — LED RGB: misturando cores

Aula 4 — ESP32: o semáforo e o RGB no celular

Para ir além

Glossário

Anexo — Checklist antes de ligar

Anexo — Tabela de cores (R, G, B)

Antes de começar: o que são Arduino e ESP32?

Um **microcontrolador** é um computador minúsculo, num único chip, feito para controlar coisas do mundo real: acender luzes, ler botões e sensores, mover motores. Ele não tem tela nem teclado — o que ele tem são **pinos**, e é você que decide, pelo programa, o que cada pino faz.

	Arduino Uno	**ESP32**
O que é	A placa mais famosa para aprender eletrônica (2005, Itália). Simples e resistente.	Chip da Espressif (China) com **Wi-Fi e Bluetooth** embutidos. Muito mais rápido e barato por função.
Velocidade / memória	16 MHz, 2 KB de RAM	240 MHz (dois núcleos), 520 KB de RAM
Tensão dos pinos	5 V	3,3 V (não ligue 5 V numa entrada!)
Pinos	14 digitais (6 com PWM ~), 6 analógicos	~25 utilizáveis, quase todos com PWM, 15+ analógicos
Para que usamos aqui	Aulas 1 a 3: aprender a lógica com calma	Aula 4: o mesmo projeto, agora controlado pelo celular
Como programar	Arduino IDE (linguagem C++ simplificada)	A **mesma** Arduino IDE, instalando o suporte ao ESP32

A boa notícia: **o jeito de pensar é o mesmo**. Um programa Arduino tem sempre duas partes: `setup()` (roda uma vez, no ligar) e `loop()` (roda para sempre, repetindo). Tudo que você aprender no Uno vale para o ESP32 — e é por isso que começamos por ele.

Como usar este guia

Cada aula tem: **materiais → montagem (figura) → código → entenda → experimente → desafios**. Os desafios são de **lógica**: a montagem quase não muda, o que muda é o que você pensa e escreve. Tente antes de olhar a resposta do professor.

Os códigos estão na pasta `aula-leds/codigo/` — todos testados.

Segurança e cuidados

- Um LED **sempre** com um resistor em série (220 Ω a 330 Ω). Sem resistor ele queima — e pode levar o pino junto.
- LED tem lado: perna **longa** = positivo (ânodo), perna **curta** = negativo (cátodo). Se não acender, inverta.
- Desligue o USB antes de mexer nos fios. Nunca ligue 5 V ou GND direto num pino configurado como saída.
- No ESP32, o potenciômetro vai no **3V3** — os pinos de entrada só aguentam 3,3 V.
- Não encoste jumpers soltos na placa ligada: curto-circuito é a causa nº 1 de placa queimada em laboratório.

Aula 1 — Ligar um LED e fazê-lo obedecer

O que você vai aprender

- O que é um pino digital de saída e como ligar/desligar 5 V nele.
- As três instruções básicas: `pinMode`, `digitalWrite`, `delay`.
- Ler um botão (`digitalRead`) e tomar decisões com `if`.
- A diferença entre **estado** (o botão está apertado?) e **evento** (o botão acabou de ser apertado?).

Materiais

- 1 Arduino Uno + cabo USB
- 1 protoboard
- 1 LED 5 mm (qualquer cor) + 1 resistor de 220 Ω
- 1 botão (push-button)
- Jumpers

Como um LED funciona

LED = *Light Emitting Diode*, diodo emissor de luz. Um diodo só deixa a corrente passar num sentido: do **ânodo (+, perna longa)** para o **cátodo (–, perna curta)**. Ele precisa de cerca de 2 V e no máximo 20 mA. Como o Arduino fornece 5 V, o **resistor** segura o excesso: (5 V − 2 V) ÷ 220 Ω ≈ 14 mA. Sem ele, a corrente dispararia e o LED (ou o pino) queimaria em segundos.

Cores do resistor 220 Ω	Vermelho	Vermelho	Marrom	Dourado
Significado	2	2	× 10	± 5 %

Montagem 1 — um LED

De	Para	Observação
Arduino pino 8	Resistor 220 Ω	qualquer lado do resistor
Outro lado do resistor	Perna **longa** do LED (+)	
Perna **curta** do LED (−)	Trilha azul (−) da protoboard	
Trilha azul (−)	Arduino GND	fecha o circuito

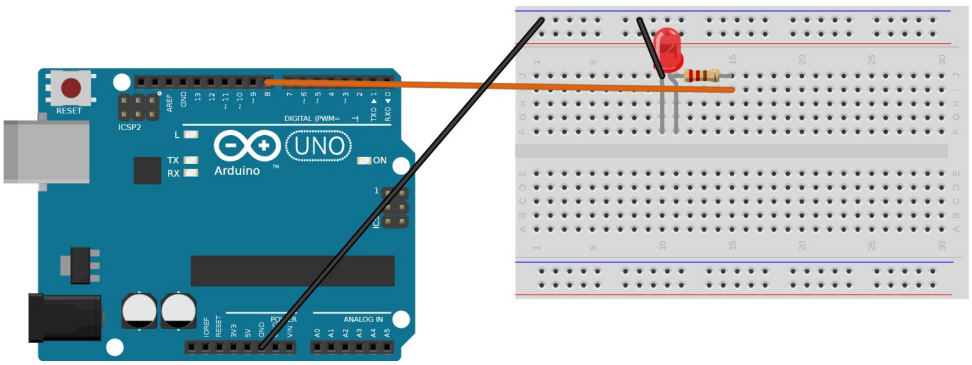


Figura 1 — Aula 1 — LED no pino 8 com resistor de 220 Ω; cátodo na trilha de GND.

Código 1 — piscar

01_blink.ino

```
/*
  01 - Primeiro LED: piscar (blink)
  Placa: Arduino Uno/Nano

  Ligacoes (na protoboard):
    pino 8 -> resistor 220 ohms -> perna LONGA do LED (anodo, +)
    perna CURTA do LED (catodo, -) -> GND

  O que aprender: pinMode define o pino como SAIDA; digitalWrite liga (HIGH = 5 V)
  ou desliga (LOW = 0 V); delay espera em milissegundos. O loop repete para sempre.
*/
const int PINO_LED = 8;

void setup() {
  pinMode(PINO_LED, OUTPUT);   // roda uma vez: o pino 8 vai ser uma saida
}

void loop() {                  // roda para sempre
  digitalWrite(PINO_LED, HIGH); // liga o LED
  delay(1000);                  // espera 1000 ms = 1 segundo
  digitalWrite(PINO_LED, LOW);  // desliga o LED
  delay(1000);
}
```

Entenda o código

- `const int PINO_LED = 8;` — dá um **nome** ao número do pino. Se trocar de pino, muda num lugar só.
- `pinMode(PINO_LED, OUTPUT)` no `setup()` — avisa que o pino vai **mandar** tensão (saída).
- `digitalWrite(PINO_LED, HIGH)` — coloca 5 V no pino; `LOW` coloca 0 V.
- `delay(1000)` — o Arduino **para tudo** por 1000 ms. Simples, mas tem um preço que veremos na Aula 2.
- O `loop()` termina e recomeça sozinho — por isso pisca para sempre.

Experimente

1. Troque os dois `delay(1000)` por `delay(100)`. E por `delay(10)`? Em que ponto você para de ver piscar? (Seu olho “funde” a partir de ~50 piscadas por segundo — é assim que o cinema funciona.)
2. Deixe ligado 900 ms e desligado 100 ms. Depois o contrário. Descreva a diferença.
3. Mude `PINO_LED` para 13 **sem** mover o fio. O que acende? (O Uno tem um LED de fábrica no pino 13.)

Desafios de lógica (LED)

1. **1.1 Batimento cardíaco** — dois piscos rápidos, uma pausa longa (tum-tum … tum-tum …).
2. **1.2 SOS em Morse** — 3 curtos, 3 longos, 3 curtos, pausa. Dica: crie duas funções, `curto()` e `longo()`, e chame cada uma 3 vezes com um `for`.
3. **1.3 Acelerando** — o LED começa piscando devagar e vai acelerando até ficar aceso “direto”, depois recomeça. Dica: uma variável `tempo` que diminui a cada volta do `loop`.

4. **1.4 Contador de piscadas** — pisque 5 vezes e pare para sempre. Dica: `for` no `setup()` e `loop()` vazio, **ou** uma variável que conta e um `if`.

Montagem 2 — LED + botão

Botão é uma **entrada**: o Arduino lê se ele está apertado. Usamos `INPUT_PULLUP`: um resistor interno mantém o pino em 5 V (`HIGH`) quando o botão está solto; ao apertar, o botão liga o pino ao GND e ele vai a `LOW`. **Apertado = 0**, solto = 1 — parece invertido, e é mesmo.

De	Para
Uma perna do botão	Arduino pino 2
Perna do outro lado do botão	Trilha azul (−) / GND
LED e resistor	iguais à montagem 1

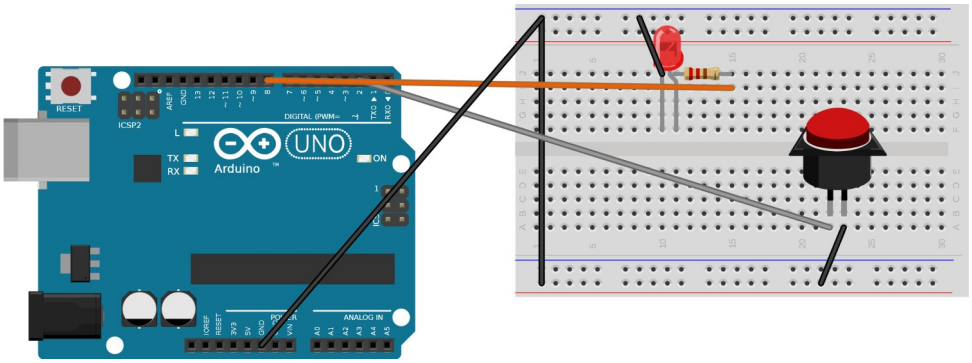


Figura 2 — Aula 1 — botão no pino 2 (INPUT_PULLUP) e LED no pino 8.

Código 2 — botão

01b_botao.ino

```
/*
  01b - Botao controlando o LED: de "interruptor" a "toggle"
  Placa: Arduino Uno/Nano

  Ligacoes:
    LED: pino 8 -> resistor 220 -> anodo (perna longa); catodo -> GND
    Botao: uma perna no pino 2, a outra no GND. Usamos INPUT_PULLUP:
           solto = HIGH (1), apertado = LOW (0)   <- atencao, e "ao contrario"!

  O que aprender: digitalRead, if/else, e a diferenca entre
  "LED acende ENQUANTO aperto" (versao A) e "cada toque inverte" (versao B).
*/
const int PINO_LED = 8;
const int PINO_BOTAO = 2;

bool ledLigado = false;
bool botaoAntes = HIGH;        // estado anterior do botao (para detectar o TOQUE)

void setup() {
  pinMode(PINO_LED, OUTPUT);
  pinMode(PINO_BOTAO, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  bool botaoAgora = digitalRead(PINO_BOTAO);   // HIGH = solto, LOW = apertado

  // ---- VERSAO A: interruptor (acende enquanto aperta) ----
  // digitalWrite(PINO_LED, botaoAgora == LOW);
```

```
  // ---- VERSAO B: toggle (cada toque inverte) ----
  // so age na BORDA: estava solto e agora esta apertado
  if (botaoAntes == HIGH && botaoAgora == LOW) {
    ledLigado = !ledLigado;                    // inverte
    digitalWrite(PINO_LED, ledLigado);
    Serial.println(ledLigado ? "LED ligado" : "LED desligado");
    delay(50);                                 // debounce: ignora "tremidas" do contato
  }
  botaoAntes = botaoAgora;
}
```

Entenda o código

- **Versão A** (comentada): `digitalWrite(PINO_LED, botaoAgora == LOW)` — o LED copia o botão. É um interruptor.
- **Versão B**: guardamos o estado **anterior** do botão. Só quando ele *estava solto e agora está apertado* (a **borda**) invertemos o LED com `ledLigado = !ledLigado`. Sem isso, enquanto o dedo ficasse no botão o LED inverteria milhares de vezes por segundo.
- `delay(50)` no fim é o **debounce**: um botão mecânico “treme” por alguns milissegundos ao ser apertado e geraria vários toques falsos.

Desafios de lógica (botão)

1. **1.5 Contador** — a cada toque, o LED pisca o número de vezes correspondente (1º toque: 1 piscada; 2º: 2; …). No 5º volta a 1.
2. **1.6 Segura para piscar** — enquanto segura o botão o LED pisca rápido; ao soltar, fica aceso.
3. **1.7 Toque duplo** — o LED só liga se você der dois toques em menos de meio segundo (dica: `millis()` guarda o instante do último toque).
4. **1.8 Reflexo** — o LED acende num momento aleatório (`random(1000, 4000)`); o aluno aperta o botão e o Arduino mostra no Serial quantos ms levou.

Aula 2 — Mini projeto: semáforo

O que você vai aprender

- Controlar várias saídas e organizar o código com **funções**.
- Pensar o semáforo como **máquina de estados** (verde → amarelo → vermelho → …).
- Por que `delay()` atrapalha e como usar `millis()` para o programa não "travar".
- Usar um botão de pedestre que funciona **a qualquer momento**.

Materiais

- Arduino Uno + protoboard
- 3 LEDs (vermelho, amarelo, verde) + 3 resistores de 220 Ω
- 1 botão
- Jumpers

Montagem

LED	Pino do Arduino	Como ligar
Vermelho	10	pino → resistor → perna longa; perna curta → GND
Amarelo	9	idem
Verde	8	idem
Botão (mais tarde)	2	outro lado no GND

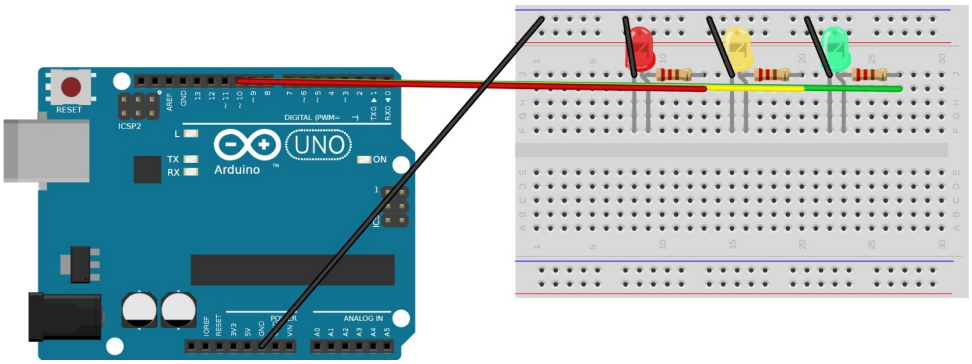


Figura 3 — Aula 2 — três LEDs, cada um com seu resistor; todos os cátodos na trilha de GND.

Código 3 — semáforo com funções

`02_semaforo.ino`

```
/*
  02 - Semaforo com 3 LEDs
  Placa: Arduino Uno/Nano

  Ligacoes (cada LED com seu resistor de 220 ohms no anodo, catodos no GND):
    pino 10 -> vermelho     pino 9 -> amarelo     pino 8 -> verde

  O que aprender: varias saidas, constantes com nome, e FUNCOES para nao
  repetir codigo. Um semaforo e uma "maquina de estados": verde -> amarelo
  -> vermelho -> verde...
*/
const int VERMELHO = 10;
```

```
const int AMARELO  = 9;
const int VERDE    = 8;

// tempos de cada fase (ms) - mexa aqui!
const int T_VERDE    = 4000;
const int T_AMARELO  = 1500;
const int T_VERMELHO = 4000;

// acende SO a cor pedida e apaga as outras
void mostrar(int cor) {
  digitalWrite(VERMELHO, cor == VERMELHO);   // true vira HIGH, false vira LOW
  digitalWrite(AMARELO,  cor == AMARELO);
  digitalWrite(VERDE,    cor == VERDE);
}

void setup() {
  pinMode(VERMELHO, OUTPUT);
  pinMode(AMARELO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  mostrar(VERDE);     delay(T_VERDE);
  mostrar(AMARELO);   delay(T_AMARELO);
  mostrar(VERMELHO);  delay(T_VERMELHO);
}
```

Entenda o código

- `mostrar(cor)` acende **só** o LED pedido: `digitalWrite(VERMELHO, cor == VERMELHO)` — a comparação vale `true` (1 = HIGH) ou `false` (0 = LOW). Três linhas resolvem todas as combinações.
- O `loop()` virou uma **lista de estados**: `mostrar(VERDE); delay(...)`. Ler o código é ler o ciclo do semáforo.
- Os tempos são **constantes com nome** no topo: mudar o semáforo é mudar números, não lógica.

Experimente

1. Faça o amarelo piscar 3 vezes antes do vermelho (semáforo “de madrugada”).
2. Inverta a ordem das fases. Um motorista entenderia? Por quê?
3. Tente acender o verde **e** o vermelho juntos usando só `mostrar()`. Não dá — por quê? O que precisaria mudar na função?

O problema do delay() e a solução com millis()

Durante `delay(4000)` o Arduino fica **surdo**: não lê botão, não recebe Serial, nada. Para um semáforo com botão de pedestre — e para o Wi-Fi da Aula 4 — isso é inaceitável. A solução é **olhar o relógio**: `millis()` devolve quantos ms se passaram desde que ligou. Guardamos quando a fase começou e, a cada volta do `loop()`, perguntamos “já deu o tempo?”. O `loop()` roda milhares de vezes por segundo e fica livre para outras tarefas.

03_semaforo_millis.ino

```
/*
  03 - Semaforo SEM delay(): usando millis()
  Placa: Arduino Uno/Nano  (mesmas ligacoes do sketch 02)

  Problema do delay(): enquanto espera, o Arduino nao faz MAIS NADA (nao le botao,
```

```
  nao responde ao Wi-Fi...). Com millis() o programa "olha o relogio" e troca de
  fase quando chega a hora, sem travar. E assim que o semaforo vai funcionar no ESP32
  junto com a pagina web.

  Extra: um botao no pino 2 (INPUT_PULLUP, outro lado no GND) e um LED de pedestre
  no pino 7 sao opcionais - veja o Desafio 2.4.
*/
const int VERMELHO = 10, AMARELO = 9, VERDE = 8;

// tabela do ciclo: qual pino acende e por quanto tempo
const int FASE_PINO[]  = { VERDE, AMARELO, VERMELHO };
const unsigned long FASE_MS[] = { 4000, 1500, 4000 };
const int NUM_FASES = 3;

int faseAtual = 0;
unsigned long inicioFase = 0;

void mostrar(int cor) {
  digitalWrite(VERMELHO, cor == VERMELHO);
  digitalWrite(AMARELO,  cor == AMARELO);
  digitalWrite(VERDE,    cor == VERDE);
}

void setup() {
  pinMode(VERMELHO, OUTPUT); pinMode(AMARELO, OUTPUT); pinMode(VERDE, OUTPUT);
  Serial.begin(9600);
  mostrar(FASE_PINO[faseAtual]);
  inicioFase = millis();
}

void loop() {
  // ja passou o tempo desta fase? entao vai para a proxima (e volta ao 0 no fim)
  if (millis() - inicioFase >= FASE_MS[faseAtual]) {
    faseAtual = (faseAtual + 1) % NUM_FASES;
    mostrar(FASE_PINO[faseAtual]);
    inicioFase = millis();
    Serial.print("fase "); Serial.println(faseAtual);
  }
  // aqui o Arduino esta LIVRE para fazer outras coisas (ler botoes, sensores, Wi-Fi...)
}
```

- `FASE_PINO[]` e `FASE_MS[]` são **vetores**: a fase 0 usa `FASE_PINO[0]` por `FASE_MS[0]` ms. Acrescentar uma fase é acrescentar um item em cada lista.
- `(faseAtual + 1) % 3` — o resto da divisão faz 0,1,2,0,1,2… É o jeito clássico de “dar a volta”.
- Repare que **não há `delay()` em lugar nenhum**.

Montagem + código 4 — botão de pedestre

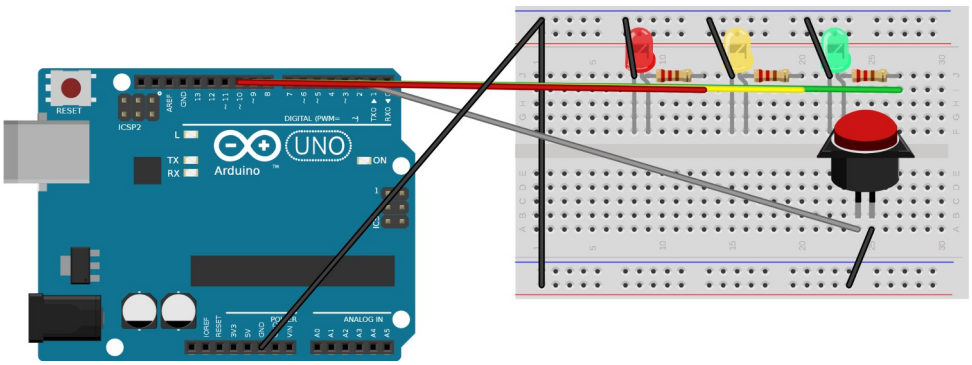


Figura 4 — Aula 2 — semáforo com botão de pedestre no pino 2.

03b_semaforo_pedestre.ino

```
/*
  03b - Semaforo com botao de pedestre (sem delay!)
  Placa: Arduino Uno/Nano

  Ligacoes: LEDs iguais ao sketch 02 (10 vermelho, 9 amarelo, 8 verde)
            Botao de pedestre: pino 2 <-> GND (INPUT_PULLUP)

  Regra: quando alguem aperta o botao, o semaforo termina a fase atual,
  passa pelo amarelo e fica no VERMELHO por um tempo extra (pedestre atravessa).
  So funciona sem travar porque usamos millis() em vez de delay().
*/
const int VERMELHO = 10, AMARELO = 9, VERDE = 8, BOTAO = 2;

const int FASE_PINO[] = { VERDE, AMARELO, VERMELHO };
const unsigned long FASE_MS[] = { 4000, 1500, 4000 };
const unsigned long T_PEDESTRE = 6000;   // vermelho mais longo quando pedido

int faseAtual = 0;
unsigned long inicioFase = 0;
bool pedidoPedestre = false;

void mostrar(int cor) {
  digitalWrite(VERMELHO, cor == VERMELHO);
  digitalWrite(AMARELO,  cor == AMARELO);
  digitalWrite(VERDE,    cor == VERDE);
}

void setup() {
  pinMode(VERMELHO, OUTPUT); pinMode(AMARELO, OUTPUT); pinMode(VERDE, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
  Serial.begin(9600);
  mostrar(FASE_PINO[0]); inicioFase = millis();
}

void loop() {
  // 1) o botao pode ser lido A QUALQUER MOMENTO, porque nao ha delay()
  if (digitalRead(BOTAO) == LOW && !pedidoPedestre) {
    pedidoPedestre = true;
    Serial.println("Pedestre pediu para atravessar");
  }

  // 2) quanto dura a fase atual? no vermelho, se houve pedido, dura mais
  unsigned long duracao = FASE_MS[faseAtual];
  if (faseAtual == 2 && pedidoPedestre) duracao = T_PEDESTRE;

  // 3) no verde, um pedido encurta a fase (vai logo para o amarelo)
  if (faseAtual == 0 && pedidoPedestre && millis() - inicioFase > 1000) duracao = 1000;

  if (millis() - inicioFase >= duracao) {
    if (faseAtual == 2) pedidoPedestre = false;   // pedido atendido ao sair do vermelho
    faseAtual = (faseAtual + 1) % 3;
    mostrar(FASE_PINO[faseAtual]);
    inicioFase = millis();
  }
}
```

O botão apenas **anota um pedido** (`pedidoPedestre = true`). Quem decide o que fazer com o pedido é a lógica das fases: encurta o verde, e estica o vermelho. Separar “registrar o evento” de “reagir ao evento” é uma ideia que você vai reusar em todo programa que responde ao mundo real.

Desafios de lógica (semáforo)

1. **2.1 Semáforo de pedestre** — acrescente um 4º LED (pedestre, pino 7) que acende **só** enquanto o dos carros está vermelho. Com o vermelho perto de acabar, ele pisca.

2. **2.2 Modo noturno** — se o botão ficar apertado por 3 s, entra em modo amarelo piscante; outro toque longo volta ao normal.
3. **2.3 Cruzamento** — dois semáforos (6 LEDs) que nunca ficam verdes ao mesmo tempo. Desenhe a tabela de estados antes de programar.
4. **2.4 Contagem regressiva no Serial** — durante o vermelho, imprima no Monitor Serial quantos segundos faltam (sem `delay`!).
5. **2.5 Sequência programável** — leia do Serial uma sequência como `VAVVA` e reproduza-a (V = verde, A = amarelo…). Dica: `Serial.read()` e um `switch`.

Aula 3 — LED RGB: misturando cores

O que você vai aprender

- O que é **PWM** e como `analogWrite()` controla o brilho.
- Que toda cor é uma mistura de vermelho, verde e azul.
- Ler um potenciômetro (`analogRead`) e converter com `map()`.
- Usar uma variável de **modo** para o mesmo hardware ter vários comportamentos.

Materiais

- Arduino Uno + protoboard
- 1 LED RGB de **cátodo comum** (4 pernas) + 3 resistores de 220 Ω
- 1 potenciômetro de 10 kΩ
- 1 botão
- Jumpers

PWM: como um pino digital faz “meio aceso”

Os pinos com ~ (3, 5, 6, 9, 10, 11) fazem **PWM**: ligam e desligam ~490 vezes por segundo. `analogWrite(pino, 64)` deixa ligado 25 % do tempo, `128` = 50 %, `255` = 100 %. O olho não vê o pisca, vê a **média**: brilho. Misturando os três brilhos do LED RGB, obtemos qualquer cor — é exatamente o que uma tela faz com cada pixel.

Montagem

Perna do LED RGB	Ligação
R (vermelho)	resistor 220 Ω → pino 9 (~)
Comum (a mais longa)	GND
G (verde)	resistor 220 Ω → pino 10 (~)
B (azul)	resistor 220 Ω → pino 11 (~)
Potenciômetro	pontas em 5V e GND; meio em A0
Botão	pino 2 e GND

Cátodo comum ou ânodo comum?
Se o seu LED RGB tiver o comum ligado ao **5 V** (ânodo comum), o brilho fica invertido: 255 apaga e 0 acende. No código, basta trocar `ANODO_COMUM = true`. Para descobrir: ligue o comum no GND e uma perna, com resistor, no 5 V — se acender, é cátodo comum.

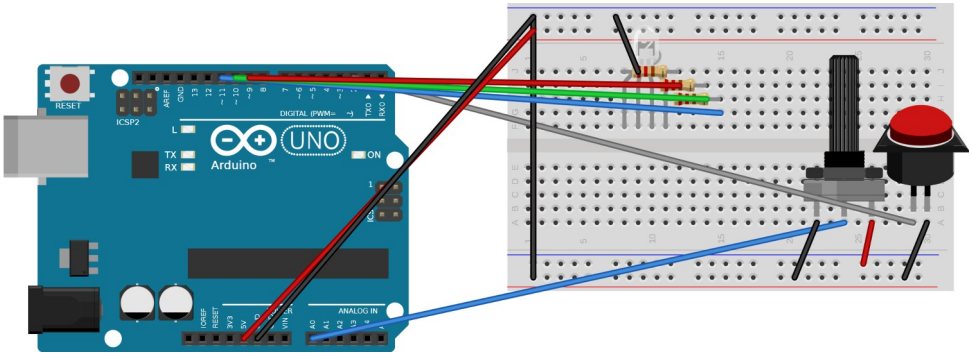


Figura 5 — Aula 3 — LED RGB (cátodo comum) nos pinos PWM 9, 10 e 11; potenciômetro em A0 e botão no pino 2.

Código 5 — cores fixas e respiração

04_rgb.ino

```
/*
  04 - LED RGB: misturando cores com PWM
  Placa: Arduino Uno/Nano

  LED RGB de CATODO COMUM (4 pernas; a mais longa e o comum -> GND):
    R -> resistor 220 -> pino 9 (~)      G -> resistor 220 -> pino 10 (~)      B -> resistor 220 -> pino
11 (~)
  Os pinos com "~" fazem PWM: analogWrite(pino, 0..255) controla o BRILHO.
  Se o seu LED for de ANODO COMUM (comum no 5V), inverta: brilho = 255 - valor.

  O que aprender: PWM, funcoes com parametros, e que qualquer cor = mistura de R, G e B.
*/
const int PINO_R = 9, PINO_G = 10, PINO_B = 11;
const bool ANODO_COMUM = false;   // troque para true se o comum for ligado ao 5V

void cor(int r, int g, int b) {   // cada valor de 0 (apagado) a 255 (maximo)
  if (ANODO_COMUM) { r = 255 - r; g = 255 - g; b = 255 - b; }
  analogWrite(PINO_R, r);
  analogWrite(PINO_G, g);
  analogWrite(PINO_B, b);
}

void setup() {
  pinMode(PINO_R, OUTPUT); pinMode(PINO_G, OUTPUT); pinMode(PINO_B, OUTPUT);
}

void loop() {
  cor(255, 0, 0);     delay(1000);   // vermelho
  cor(0, 255, 0);     delay(1000);   // verde
  cor(0, 0, 255);     delay(1000);   // azul
  cor(255, 255, 0);   delay(1000);   // amarelo  (R + G)
  cor(0, 255, 255);   delay(1000);   // ciano    (G + B)
  cor(255, 0, 255);   delay(1000);   // magenta  (R + B)
  cor(255, 255, 255); delay(1000);   // branco
  cor(0, 0, 0);       delay(500);    // apagado

  // "respiracao": o vermelho sobe e desce devagar
  for (int i = 0; i <= 255; i += 5) { cor(i, 0, 0); delay(20); }
  for (int i = 255; i >= 0; i -= 5) { cor(i, 0, 0); delay(20); }
}
```

Entenda o código

- `cor(r, g, b)` recebe três **parâmetros**. Uma função com parâmetros é uma “receita” que aceita ingredientes diferentes.

- Amarelo = vermelho + verde; ciano = verde + azul; magenta = vermelho + azul; branco = tudo. Não existe “pigmento amarelo” no LED — é a **luz** se somando.
- O `for` de 0 a 255 de 5 em 5 faz o **fade** (respiração). Mude o passo e o `delay` e sinta a diferença.

Código 6 — potenciômetro escolhe a cor, botão escolhe o modo

04b_rgb_pot.ino

```
/*
  04b - LED RGB controlado por potenciometro (cor) e botao (modo)
  Placa: Arduino Uno/Nano

  Ligacoes: RGB igual ao sketch 04 (R->9, G->10, B->11, comum->GND, resistores 220)
            Potenciometro 10k: pontas em 5V e GND, meio em A0
            Botao: pino 2 <-> GND (INPUT_PULLUP)

  O potenciometro escolhe a COR (girando pelo arco-iris) e o botao troca o MODO:
    modo 0: cor fixa escolhida pelo pot
    modo 1: pot controla o BRILHO do branco
    modo 2: arco-iris automatico (pot controla a velocidade)

  O que aprender: analogRead + map, funcao que converte "matiz" (0..767) em R,G,B,
  e uma variavel de MODO para o mesmo hardware fazer coisas diferentes.
*/
const int PINO_R = 9, PINO_G = 10, PINO_B = 11, PINO_POT = A0, BOTAO = 2;
int modo = 0;
bool botaoAntes = HIGH;

void cor(int r, int g, int b) { analogWrite(PINO_R, r); analogWrite(PINO_G, g); analogWrite(PINO_B, b); }

// matiz de 0 a 767: vermelho -> verde -> azul -> vermelho
void corPorMatiz(int h) {
  h = h % 768;
  if (h < 256)       cor(255 - h, h, 0);
  else if (h < 512)  cor(0, 511 - h, h - 256);
  else               cor(h - 512, 0, 767 - h);
}

void setup() {
  pinMode(PINO_R, OUTPUT); pinMode(PINO_G, OUTPUT); pinMode(PINO_B, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  // botao troca de modo (na borda, com debounce)
  bool botaoAgora = digitalRead(BOTAO);
  if (botaoAntes == HIGH && botaoAgora == LOW) {
    modo = (modo + 1) % 3;
    Serial.print("modo "); Serial.println(modo);
    delay(50);
  }
  botaoAntes = botaoAgora;

  int pot = analogRead(PINO_POT);                // 0..1023

  if (modo == 0) {
    corPorMatiz(map(pot, 0, 1023, 0, 767));      // pot escolhe a cor
  } else if (modo == 1) {
    int brilho = map(pot, 0, 1023, 0, 255);      // pot escolhe o brilho do branco
    cor(brilho, brilho, brilho);
  } else {
    int velocidade = map(pot, 0, 1023, 1, 40);   // pot escolhe a velocidade
    corPorMatiz((millis() / 10) * velocidade / 10);
  }
  delay(10);
}
```

- `corPorMatiz(h)` percorre o **arco-íris** com um número só (0 a 767): enquanto uma cor desce, a próxima sobe. Desenhe o gráfico das três curvas — é o círculo cromático "desenrolado".
- `map(pot, 0, 1023, 0, 767)` converte a leitura do pot na matiz.
- A variável `modo` (0, 1, 2) muda o comportamento inteiro do `loop()` — é assim que um aparelho de verdade tem "funções".

Desafios de lógica (RGB)

1. **3.1 Termômetro de cor** — o pot vai de azul (frio) a vermelho (quente) passando pelo branco, sem verde. Dica: duas retas, não três.
2. **3.2 Semáforo num só LED** — refaça o semáforo da Aula 2 usando **apenas** o LED RGB. O amarelo é (255, 60, 0), não (255, 255, 0) — teste e explique.
3. **3.3 Dado de cores** — a cada toque no botão sorteia uma de 6 cores (`random(6)`) com um "rufar" rápido antes de parar.
4. **3.4 Memória** — o botão grava a cor atual do pot; 3 toques gravam 3 cores; depois o LED alterna entre elas. (Prévia do "gravar e reproduzir" de outros cursos.)
5. **3.5 Lâmpada de humor** — `modo 2` com transição suave entre cores aleatórias: a cada 3 s sorteia uma cor e vai até ela em 1 s, interpolando.

Aula 4 — ESP32: o semáforo e o RGB no celular

O que você vai aprender

- Migrar um projeto do Uno para o ESP32 — o que muda, o que não muda.
- PWM no ESP32 com `ledcAttach`/`ledcWrite`.
- O que é uma rede Wi-Fi, um endereço IP e um **servidor web**.
- Escrever uma página HTML mínima que **conversa** com o ESP32 (JavaScript `fetch`).

O que muda do Uno para o ESP32

Item	Uno	ESP32	No código
Tensão	5 V	3,3 V	LEDs um pouco menos brilhantes; 220 Ω continua ok. Pot no 3V3.
PWM	`analogWrite` nos pinos ~	qualquer GPIO, via LEDC	`ledcAttach(pino, 5000, 8)` no setup; `ledcWrite(pino, 0..255)`
ADC	0..1023	0..4095	`analogReadResolution(12)` e `map(..., 0, 4095, ...)`
Serial	9600	115200	ajustar o Monitor Serial
Pinos usados	8, 9, 10 / 9, 10, 11 / 2 / A0	25, 26, 27 / 14, 12, 13 / 4 / 34	só constantes no topo
Wi-Fi	não tem	tem	`#include <WiFi.h>`, `<WebServer.h>`

Materiais

- ESP32 Dev Module + cabo micro-USB **de dados**
- Protoboard grande (830 pontos)
- 3 LEDs + LED RGB + 6 resistores 220 Ω
- 1 botão + 1 potenciômetro 10 kΩ
- Celular ou notebook com Wi-Fi

Preparando a IDE

1. Arquivo → Preferências → URLs adicionais: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
2. Gerenciador de Placas → **esp32 by Espressif** → Placa: **ESP32 Dev Module**.
3. Se a porta não aparecer: driver CP210x/CH340 (professor). Se “Failed to connect”: segure o botão **BOOT** ao iniciar o upload.

Montagem

Componente	GPIO	Observação
LED vermelho / amarelo / verde	25 / 26 / 27	resistor 220 Ω em cada

Componente	GPIO	Observação
RGB: R / G / B	14 / 12 / 13	comum no GND
Botão	4 ↔ GND	INPUT_PULLUP
Potenciômetro	34 (meio); pontas em **3V3** e GND	GPIO 34 é só entrada, perfeito para ADC

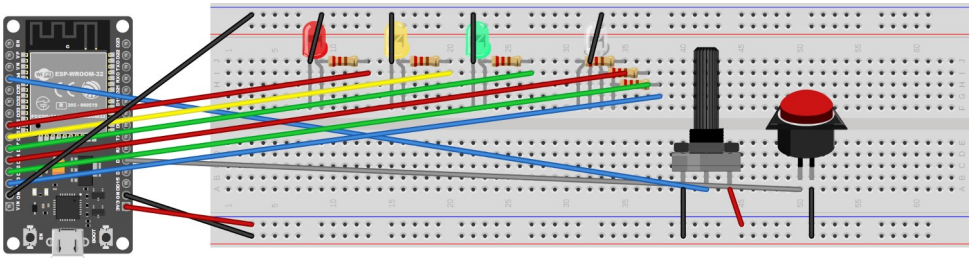


Figura 6 — Aula 4 — semáforo, LED RGB, botão e potenciômetro no ESP32. Trilha inferior: 3V3 do ESP32 (pot); GND comum nas duas trilhas.

Código 7 — semáforo + RGB no ESP32 (ainda sem Wi-Fi)

05_esp32_semaforo_rgb.ino

```
/*
  05 - Semaforo + LED RGB no ESP32 (sem Wi-Fi ainda)
  Placa: ESP32 Dev Module (core esp32 3.x)

  Ligacoes (resistor de 220 ohms em serie com cada LED; catodos/comum no GND):
    Semaforo: GPIO 25 -> vermelho   GPIO 26 -> amarelo   GPIO 27 -> verde
    RGB:      GPIO 14 -> R          GPIO 12 -> G         GPIO 13 -> B
    Botao:    GPIO 4 <-> GND (INPUT_PULLUP)  -> avanca o semaforo na mao
    Pot 10k:  pontas em 3V3 e GND, meio em GPIO 34 -> escolhe a cor do RGB

  O que muda em relacao ao Arduino Uno:
    1. Os pinos sao 3,3 V (o LED acende um pouco mais fraco; 220 ohms continua bom).
    2. analogWrite() existe no core 3.x, mas o "jeito ESP32" e o LEDC:
         ledcAttach(pino, frequencia, resolucao) e ledcWrite(pino, valor).
       Qualquer GPIO de saida pode fazer PWM (no Uno so os pinos com "~").
    3. Nada de delay(): usamos millis() para o semaforo, pois no sketch 06 o Wi-Fi
       precisa do loop() livre.
*/
const int VERMELHO = 25, AMARELO = 26, VERDE = 27;
const int PINO_R = 14, PINO_G = 12, PINO_B = 13;
const int BOTAO = 4, PINO_POT = 34;

const int FASE_PINO[] = { VERDE, AMARELO, VERMELHO };
const unsigned long FASE_MS[] = { 4000, 1500, 4000 };
int faseAtual = 0;
unsigned long inicioFase = 0;
bool botaoAntes = HIGH;

void proximaFase() {
  faseAtual = (faseAtual + 1) % 3;
  mostrar(FASE_PINO[faseAtual]);
  inicioFase = millis();
}

void mostrar(int corSemaforo) {
  digitalWrite(VERMELHO, corSemaforo == VERMELHO);
  digitalWrite(AMARELO,  corSemaforo == AMARELO);
  digitalWrite(VERDE,    corSemaforo == VERDE);
}

void cor(int r, int g, int b) {
  ledcWrite(PINO_R, r);
  ledcWrite(PINO_G, g);
  ledcWrite(PINO_B, b);
```

```
}

void corPorMatiz(int h) {            // 0..767: vermelho -> verde -> azul -> vermelho
  h = h % 768;
  if (h < 256)       cor(255 - h, h, 0);
  else if (h < 512)  cor(0, 511 - h, h - 256);
  else               cor(h - 512, 0, 767 - h);
}

void setup() {
  Serial.begin(115200);
  pinMode(VERMELHO, OUTPUT); pinMode(AMARELO, OUTPUT); pinMode(VERDE, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
  analogReadResolution(12);        // pot: 0..4095
  ledcAttach(PINO_R, 5000, 8);     // 5 kHz, 8 bits (0..255) - igual ao analogWrite do Uno
  ledcAttach(PINO_G, 5000, 8);
  ledcAttach(PINO_B, 5000, 8);
  mostrar(FASE_PINO[0]);
  inicioFase = millis();
}

void loop() {
  // semaforo por millis() - ou pelo botao
  if (millis() - inicioFase >= FASE_MS[faseAtual]) proximaFase();
  bool botaoAgora = digitalRead(BOTAO);
  if (botaoAntes == HIGH && botaoAgora == LOW) { proximaFase(); delay(50); }
  botaoAntes = botaoAgora;

  // RGB: o potenciometro escolhe a cor (matiz)
  corPorMatiz(map(analogRead(PINO_POT), 0, 4095, 0, 767));
}
```

- Compare com os sketches 03b e 04b: a **lógica é idêntica**; mudaram só os números dos pinos e as três linhas de PWM/ADC. Migrar de plataforma é isso.
- O botão avança o semáforo “na mão” e o pot escolhe a cor do RGB — o hardware da Aula 4 já testa tudo antes do Wi-Fi.

Como funciona o controle pela web

1. O ESP32 **cria a própria rede Wi-Fi** (`Semaforo-ESP32`, senha `12345678`) — modo *Access Point*. Não precisa da rede da escola.
2. Você conecta o celular nessa rede e abre `http://192.168.4.1` no navegador.
3. O ESP32 responde com uma **página HTML** (guardada na memória dele, `PAGINA[]`).
4. Cada botão da página chama um endereço: `/semaforo?cor=verde` ou `/rgb?r=255&g=0&b=0`. Isso é um **pedido HTTP**, igual ao que seu navegador faz num site.
5. O ESP32 lê os parâmetros (`servidor.arg("cor")`), acende os LEDs e responde “ok”. O JavaScript da página (`fetch`) faz isso sem recarregar a tela.

06_esp32_web.ino

```
/*
  06 - Controle pelo Wi-Fi: pagina web local para o semaforo e o LED RGB
  Placa: ESP32 Dev Module (core esp32 3.x)   Ligacoes: iguais ao sketch 05

  Como funciona:
    1. O ESP32 entra na rede Wi-Fi (ou cria a sua propria, se MODO_AP = true).
    2. Ele roda um SERVIDOR WEB: quem acessar http://<ip>/ recebe a pagina HTML abaixo.
    3. Os botoes da pagina chamam enderecos como /semaforo?cor=verde  e  /rgb?r=255&g=0&b=0
    4. O ESP32 le os parametros e acende os LEDs. Tudo sem delay(), para nunca travar o servidor.

  Abra o Monitor Serial (115200) para ver o endereco IP. Celular e ESP32 na MESMA rede.
```

```
*/
#include <WiFi.h>
#include <WebServer.h>

// ---------- rede ----------
const bool MODO_AP = true;                 // true: o ESP32 cria a rede "Semaforo-ESP32" (senha abaixo)
const char* SSID_AP = "Semaforo-ESP32";    // false: entra na rede da escola (preencha abaixo)
const char* SENHA_AP = "12345678";
const char* SSID_ESCOLA = "NOME-DA-REDE";
const char* SENHA_ESCOLA = "SENHA-DA-REDE";

// ---------- pinos ----------
const int VERMELHO = 25, AMARELO = 26, VERDE = 27;
const int PINO_R = 14, PINO_G = 12, PINO_B = 13;
const int BOTAO = 4, PINO_POT = 34;        // botao fisico: avanca o semaforo; pot: brilho geral do RGB
bool botaoAntes = HIGH;

WebServer servidor(80);

// estado atual (o que a pagina mostra e o que os LEDs fazem)
String corSemaforo = "auto";               // "auto", "verde", "amarelo", "vermelho", "off"
int corR = 0, corG = 0, corB = 128;
const int FASE_PINO[] = { VERDE, AMARELO, VERMELHO };
const unsigned long FASE_MS[] = { 4000, 1500, 4000 };
int faseAtual = 0;
unsigned long inicioFase = 0;

// ---------- pagina HTML (fica na memoria flash) ----------
const char PAGINA[] PROGMEM = R"HTML(
<!DOCTYPE html><html lang="pt-br"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1">
<title>Semaforo ESP32</title>
<style>
 body{font-family:sans-serif;background:#111;color:#eee;text-align:center;margin:0;padding:16px}
 h1{font-size:1.4em} .card{background:#222;border-radius:12px;padding:16px;margin:12px auto;max-
width:360px}
 button{font-size:1.1em;padding:12px 18px;margin:6px;border:0;border-radius:8px;color:#fff;cursor:pointer}
 .vd{background:#2a2} .am{background:#ca0} .vm{background:#c22} .au{background:#448} .off{background:#555}
 input[type=color]{width:120px;height:60px;border:0;background:none}
 #estado{opacity:.8}
</style></head><body>
<h1>Semaforo + RGB pelo Wi-Fi</h1>
<div class="card"><h2>Semaforo</h2>
 <button class="vd" onclick="sem('verde')">Verde</button>
 <button class="am" onclick="sem('amarelo')">Amarelo</button>
 <button class="vm" onclick="sem('vermelho')">Vermelho</button><br>
 <button class="au" onclick="sem('auto')">Automatico</button>
 <button class="off" onclick="sem('off')">Apagar</button>
 <p id="estado">estado: %SEMAFORO%</p></div>
<div class="card"><h2>LED RGB</h2>
 <input type="color" id="cor" value="%COR%" oninput="rgb(this.value)">
 <p>toque na cor para escolher</p></div>
<script>
 function sem(c){fetch('/semaforo?
cor='+c).then(r=>r.text()).then(t=>document.getElementById('estado').textContent='estado: '+t)}
 function rgb(hex){const
r=parseInt(hex.substr(1,2),16),g=parseInt(hex.substr(3,2),16),b=parseInt(hex.substr(5,2),16);
   fetch('/rgb?r='+r+'&g='+g+'&b='+b)}
</script></body></html>
)HTML";

// ---------- LEDs ----------
void mostrar(int pino) {
  digitalWrite(VERMELHO, pino == VERMELHO);
  digitalWrite(AMARELO,  pino == AMARELO);
  digitalWrite(VERDE,    pino == VERDE);
}
void aplicarSemaforo() {
  if (corSemaforo == "verde")         mostrar(VERDE);
  else if (corSemaforo == "amarelo")  mostrar(AMARELO);
  else if (corSemaforo == "vermelho") mostrar(VERMELHO);
  else if (corSemaforo == "off")      mostrar(-1);
  // "auto": quem cuida e o loop() com millis()
}
```

```
int brilho = 255;                                   // 0..255, vem do potenciometro
void cor(int r, int g, int b) {                     // aplica o brilho do pot a cor escolhida na pagina
  ledcWrite(PINO_R, r * brilho / 255);
  ledcWrite(PINO_G, g * brilho / 255);
  ledcWrite(PINO_B, b * brilho / 255);
}

// ---------- rotas do servidor ----------
void paginaInicial() {
  String html = FPSTR(PAGINA);
  html.replace("%SEMAFORO%", corSemaforo);
  char hex[8]; snprintf(hex, sizeof(hex), "#%02x%02x%02x", corR, corG, corB);
  html.replace("%COR%", hex);
  servidor.send(200, "text/html; charset=utf-8", html);
}
void rotaSemaforo() {
  if (servidor.hasArg("cor")) corSemaforo = servidor.arg("cor");
  aplicarSemaforo();
  Serial.println("semaforo -> " + corSemaforo);
  servidor.send(200, "text/plain", corSemaforo);
}
void rotaRgb() {
  corR = servidor.arg("r").toInt();
  corG = servidor.arg("g").toInt();
  corB = servidor.arg("b").toInt();
  cor(corR, corG, corB);
  Serial.printf("rgb -> %d %d %d\n", corR, corG, corB);
  servidor.send(200, "text/plain", "ok");
}

void setup() {
  Serial.begin(115200);
  pinMode(VERMELHO, OUTPUT); pinMode(AMARELO, OUTPUT); pinMode(VERDE, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
  analogReadResolution(12);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);
  cor(corR, corG, corB);

  if (MODO_AP) {
    WiFi.softAP(SSID_AP, SENHA_AP);
    Serial.print("Rede criada: "); Serial.print(SSID_AP);
    Serial.print("  ->  acesse http://"); Serial.println(WiFi.softAPIP());
  } else {
    WiFi.begin(SSID_ESCOLA, SENHA_ESCOLA);
    Serial.print("Conectando");
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.print("\nConectado!  ->  acesse http://"); Serial.println(WiFi.localIP());
  }

  servidor.on("/", paginaInicial);
  servidor.on("/semaforo", rotaSemaforo);
  servidor.on("/rgb", rotaRgb);
  servidor.begin();
  inicioFase = millis();
}

void loop() {
  servidor.handleClient();                      // atende quem acessou a pagina
  if (corSemaforo == "auto" && millis() - inicioFase >= FASE_MS[faseAtual]) {
    faseAtual = (faseAtual + 1) % 3;
    mostrar(FASE_PINO[faseAtual]);
    inicioFase = millis();
  }
  // botao fisico: forca a proxima cor (a pagina passa a mostrar esse estado)
  bool botaoAgora = digitalRead(BOTAO);
  if (botaoAntes == HIGH && botaoAgora == LOW) {
    if (corSemaforo == "verde") corSemaforo = "amarelo";
    else if (corSemaforo == "amarelo") corSemaforo = "vermelho";
    else corSemaforo = "verde";
    aplicarSemaforo();
    delay(50);
  }
  botaoAntes = botaoAgora;
  // potenciometro: brilho geral do RGB (le a cada 100 ms para nao "tremer")
```

```
  static unsigned long ultimoPot = 0;
  if (millis() - ultimoPot > 100) {
    ultimoPot = millis();
    int novo = map(analogRead(PINO_POT), 0, 4095, 0, 255);
    if (abs(novo - brilho) > 3) { brilho = novo; cor(corR, corG, corB); }
  }
}
```

Entenda o código

- `servidor.on("/semaforo", rotaSemaforo)` — “quando alguém pedir /semaforo, chame esta função”. Cada endereço é uma **rota**.
- `servidor.handleClient()` no `loop()` atende os pedidos. Como o semáforo usa `millis()`, o `loop()` nunca trava e o site responde na hora. **Se houvesse um `delay(4000)`, a página levaria 4 s para responder.**
- A página troca `%SEMAFORO%` e `%COR%` pelo estado atual antes de enviar — assim, ao abrir, ela já mostra o que está acontecendo.
- O botão físico e o pot continuam funcionando: o botão muda `corSemaforo`, e a página mostra o novo estado na próxima atualização.
- `R"HTML( ... )HTML"` é um jeito de escrever um texto grande com aspas dentro sem escapar nada.

Desafios de lógica (web)

1. **4.1 Estado ao vivo** — faça a página atualizar o texto “estado:” sozinha a cada 1 s (dica: `setInterval` + uma rota `/estado` que responde só o texto).
2. **4.2 Botões de cor rápida** — 4 botões na página (vermelho, verde, azul, apagar) além do seletor de cor.
3. **4.3 Tempo do semáforo pela web** — um campo numérico que muda `FASE_MS[0]` (tempo do verde). Rota `/tempo?verde=6000`.
4. **4.4 Quem controla?** — se alguém mexer no pot nos últimos 5 s, o pot manda; senão, a página manda. (Conflito entre controle local e remoto — problema real de automação.)
5. **4.5 Dois ESP32** — um ESP32 controla o semáforo do outro pela rede (dica: `HTTPClient` fazendo `GET http://192.168.4.1/semaforo?cor=...`).

Para ir além

- Trocar os LEDs por uma **fita de LEDs endereçáveis** (WS2812/NeoPixel): cada LED tem sua cor, com a mesma lógica de RGB.
- Mandar o estado do semáforo por **MQTT** para o painel do laboratório (Node-RED/Grafana).
- Um **sensor de luz** (LDR) que liga o modo noturno sozinho.
- Montar um semáforo de verdade com LEDs de alto brilho e um transistor por cor.

Glossário

Termo	Significado
Pino digital	Saída ou entrada que só conhece dois valores: HIGH (5 V / 3,3 V) e LOW (0 V).
PWM	Ligar e desligar muito rápido para simular um valor intermediário (brilho, velocidade).
ADC / analogRead	Converte uma tensão (do pot) num número: 0..1023 no Uno, 0..4095 no ESP32.
INPUT_PULLUP	Entrada com resistor interno para 5 V: botão solto = 1, apertado = 0.
Borda	O instante em que um sinal muda (solto → apertado). Diferente de estado (está apertado).
Debounce	Ignorar as “tremidas” elétricas de um botão mecânico.
Máquina de estados	Programa organizado em estados (verde, amarelo…) e regras de transição.
millis()	Relógio interno em ms; permite esperar sem travar.
Access Point	O ESP32 cria a sua própria rede Wi-Fi.
IP	Endereço numérico de um aparelho na rede (ex.: 192.168.4.1).
HTTP / rota	Pedido do navegador a um endereço; a rota é a função que responde a ele.
HTML / JavaScript	Linguagem da página (o que aparece) e da interação (o que acontece ao clicar).

Anexo — Checklist antes de ligar

- [ ] Todo LED tem resistor em série
- [ ] Perna longa (+) no lado do pino/resistor; perna curta (−) no GND
- [ ] Botões com um lado no GND e `INPUT_PULLUP` no código
- [ ] Pot: pontas em 5V/GND (Uno) ou **3V3**/GND (ESP32); meio no pino analógico
- [ ] GND da placa ligado à trilha azul da protoboard
- [ ] Placa certa selecionada na IDE (Uno / ESP32 Dev Module) e porta COM correta
- [ ] Monitor Serial fechado antes de gravar (ele pode segurar a porta)

Anexo — Tabela de cores (R, G, B)

Cor	R	G	B
Vermelho	255	0	0
Laranja	255	60	0
Amarelo	255	150	0
Verde	0	255	0
Ciano	0	255	255
Azul	0	0	255
Roxo	128	0	255
Magenta	255	0	255
Rosa	255	40	80
Branco	255	255	255